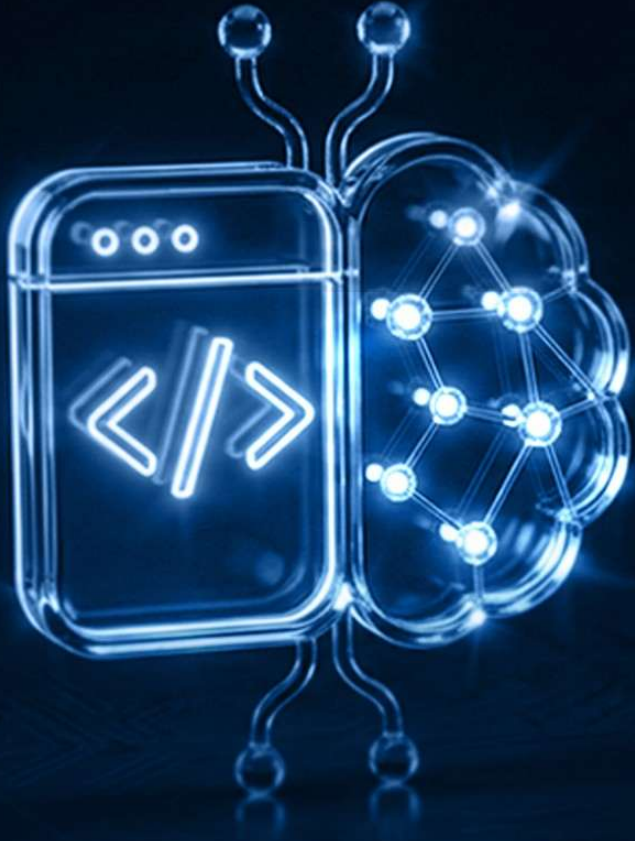


دوره آموزشی

# GenAI

مهندسی نرم افزار در عصر هوش مصنوعی

مدرس: **علیرضا ارومند**



# جلسه اول

تغییر پارادایم در مهندسی نرم افزار



# چه خواهیم آموخت

- محدودیت‌ها و چالش‌های Vibe-Coding در پروژه‌های واقعی
- چرا نیاز داریم از Vibe-Coding به SDD حرکت کنیم
- مفاهیم کلیدی SDD و اصول آن
- نقش Specification به عنوان مرکز فرایند توسعه
- ساختار و اجزای یک Specification قابل اجرا
- فرایند عملی SDD و Pipeline پیشنهادی
- نقاط کنترل کیفیت و Review Gates
- Constitution و محدودیت‌های منفی برای کنترل Agent
- تست‌های مبتنی بر ویژگی و تضمین کیفیت در SDD
- جمع‌بندی و نگاه به مهندس نرم‌افزار در عصر AI



# از وایب‌کدینگ تا توسعه مبتنی بر مشخصات

...



یک تغییر رویکرد بزرگ  
در مهندسی نرم‌افزار



معماری نیت،  
پیش از معماری کد



حرکت از تولید سریع  
به تولید کنترل‌شده



نقش جدید انسان  
در عصر هوشواره



این سفر، ما را از نوشتن کد به توصیف نیت می‌رساند.

# کمیاب‌ترین منبع دیگر کد نیست

...



**تولید کد سریع‌تر  
و ارزان‌تر شده است**

ابزارها و مدل‌های هوشمند،  
فرآیند نوشتن کد را متحول کرده‌اند.



**سرعت تولید، تضمین‌کننده  
کیفیت مهندسی نیست**

کد بیشتر همیشه به معنی راه‌حل  
بهتر یا پایدارتر نیست.



**هوشواره بخش بزرگی  
از پیاده‌سازی را انجام می‌دهد**

از ساخت قطعه‌ها تا تکمیل توابع و  
رفع اشکال‌های رایج.



**ابهام اکنون مهم‌تر از  
محدودیت فنی است**

وقتی مسئله مبهم باشد، حتی قوی‌ترین  
هوشواره‌ها مسیر غلط می‌روند.



**امروز مسئله اصلی، شفاف‌سازی نیت و کاهش ابهام است.**

# گلوگاه مهندسی در حال جابه‌جایی است

## گلوگاه قدیمی

نوشتن و دیباگ کردن کد

نوشتن کد </>

دیباگ کردن

زمان‌بر و خسته‌کننده

تمرکز روی تولید کد و رفع خطاهای فنی

## گلوگاه جدید

بیان دقیق نیت

### بیان دقیق نیت

- هدف چیست؟
- چه کارهایی باید انجام شود؟
- چه محدودیت‌هایی داریم؟
- چه نتیجه‌ای انتظار داریم؟

تمرکز روی شفاف‌سازی نیت و تصمیم‌های کلیدی



مسئله اصلی دیگر «چگونه بسازیم» تست



مسئله اصلی «دقیقاً چه می‌خواهیم» است



# وایب‌کدینگ چیست؟

توسعه با رویکرد بداهه‌محور و بدون مشخصات پایدار



وایب‌کدینگ برای اکتشاف و شروع سریع عالی است، اما جایگزین مهندسی مبتنی بر مشخصات نیست.



**توسعه با درخواست‌های کلی و بداهه‌محور آغاز می‌شود**  
توسعه با ایده‌ها و خواسته‌های عمومی و بدون برنامه‌ریزی جزئیات در حین گفتگو شکل می‌گیرد.



**پرامیت عملاً جای مشخصات را می‌گیرد**  
به جای سند مشخصات، چند جمله یا درخواست گفتاری جهت سیستم را تعیین می‌کند.



**خروجی اجرا و به صورت تکراری اصلاح می‌شود**  
نتیجه اجرا می‌شود، اشکالات دیده می‌شود و دوباره درخواست تغییر ارسال می‌گردد.



**نیت به شکل پایدار ثبت نمی‌شود**  
تصمیم‌ها، فرضیات و اهداف کلیدی به صورت رسمی و پایدار مستند و قابل اتکا ذخیره نمی‌شوند.

# چرا وایبکدینگ جذاب است؟



صفحه خالی را  
به سرعت از بین می برد

با چند کلمه، از ایده به اولین نسخه  
قابل مشاهده می رسید.



اکتشاف ایده ها را  
ساده تر می کند

امکان امتحان سریع ایده های مختلف بدون  
پیچیدگی فنی.



بازخورد قابل مشاهده را  
سریع فراهم می کند

نتیجه کار را بلافاصله می بینید و بهتر  
تصمیم می گیرید.



فاصله میان تصور و  
نمونه اولیه را کاهش می دهد

از فکر کردن به ساختن، در کمترین زمان  
ممکن.



وایبکدینگ شروع را سریع می کند و مسیر رسیدن از ایده به نمونه اولیه را کوتاه تر می سازد.



# مرز کارایی وایب کدینگ



برای نمونه‌سازی و اکتشاف  
بسیار مؤثر است



در سیستم‌های کوچک،  
زمینه قابل مدیریت است



با افزایش مقیاس،  
فرضیات پنهان بیشتر می‌شوند



سیستم‌های حساس به کنترل  
مهندسی نیاز دارند



وایب کدینگ برای شروع عالی است، اما در مقیاس بالا به کنترل مهندسی نیاز داریم.

# چهار محدودیت ساختاری وایب کدینگ

1

## فرضیات پنهان و تصمیم‌های حدسی



بخش مهمی از تصمیم‌ها گفته  
نمی‌شود و مدل آن‌ها را حدس می‌زند

2

## پوسیدگی زمینه در پروژه‌های بلندمدت



با گذشت زمان، زمینه و تصمیم‌های  
کلیدی کم‌رنگ می‌شوند

3

## انحراف تدریجی از معماری سیستم



بهینه‌سازی‌های محلی می‌توانند  
سیستم را از معماری اصلی دور کنند

4

## فقدان ردیابی و پاسخ‌گویی مهندسی



منشأ تصمیم‌ها روشن نمی‌ماند و  
ارزیابی مهندسی دشوار می‌شود



این چهار محدودیت نشان می‌دهند چرا فرایند بداهه‌محور در مقیاس واقعی **ناپایدار** است.

7

# خطر فرضیات پنهان

وقتی تصمیم‌ها در پرامپت‌ها بیان نمی‌شوند، مدل آن‌ها را حدس می‌زند؛  
و حدس، جای تصمیم آگاهانه را نمی‌گیرد.

## پرامپت شما

یک سیستم سفارش آنلاین  
با پرداخت بساز

آنچه شما می‌گویید  
(مشخص و آشکار)

## فرضیات پنهان

- معماری سیستم؟
- مدل داده؟
- قوانین کسب‌وکار؟
- امنیت و مجوزها؟
- تجربه کاربری؟
- سایر تصمیم‌ها؟

آنچه مدل حدس می‌زند  
(حدس‌ها و پرکردن جاهای خالی)

1 پرامپت‌ها معمولاً تمام تصمیم‌ها را بیان نمی‌کنند  
بسیاری از تصمیم‌های کلیدی در گفت‌وگو گفته نمی‌شوند  
یا به‌طور ضمنی فرض می‌گردند.



2 مدل جاهای خالی را با احتمال پر می‌کند  
مدل با تکیه بر آموزش و الگوها، گزینه‌ای را انتخاب می‌کند  
که از نظر آماری محتمل‌تر است.



3 تصمیم محتمل الزاماً تصمیم سازمانی نیست  
آنچه محتمل است، ممکن است با سیاست‌ها، استانداردها  
و اهداف سازمان هم‌راستا نباشد.



4 خروجی می‌تواند بی‌سروصدا اشتباه باشد  
خطا همیشه آشکار نیست؛ خروجی ممکن است کار کند،  
اما پایه‌های اشتباهی داشته باشد.



**خطر اصلی:** اشتباهات ناشی از فرضیات پنهان، تا دیرنگام آشکار نمی‌شوند و هزینه‌ی اصلاح آن‌ها بسیار بالاست.

# پوسیدگی زمینه

زمینه از بین نمی‌رود، اما در استفاده طولانی‌مدت، فرسوده می‌شود.



**ظرفیت زمینه با توجه مؤثر پکسان نیست**  
مدل محدودیتی در نگهداری مؤثر زمینه دارد، نه صرفاً حجم آن.



**تصمیم‌های بنیادین در طول زمان کمرنگ می‌شوند**  
جزئیات جدید، تصمیم‌های اولیه را از مرکز توجه خارج می‌کنند.



**مدل مسائل محلی را جداگانه بهینه می‌کند**  
هر درخواست به صورت مستقل حل می‌شود، نه در چارچوب کل سیستم.



**مجموعه تصمیم‌های محلی، سیستم منسجم نمی‌سازد**  
بهینه‌سازی محلی لزوماً به انسجام سراسری منجر نمی‌شود.

## در ابتدای پروژه

زمینه کامل و زنده



درک شفاف، تصمیم‌های هم‌راستا، سیستم منسجم

## در طول زمان

زمینه فرسوده و ناقص



درک مبهم، تصمیم‌های ناسازگار، سیستم ناپوسته



پوسیدگی زمینه به این معناست که «دانستن زیاد» الزاماً به «از یاد نرفتن» منجر نمی‌شود.



# فرسایش معماری و پاسخ‌گویی

وقتی توسعه براساس گفت‌وگوهای لحظه‌ای پیش می‌رود، معماری و پاسخ‌گویی به مرور فرسایش می‌یابند.



**1** سریع‌ترین راه‌حل ممکن است معماری را نقض کند  
انتخاب راه‌حل سریع بدون در نظر گرفتن اصول و مرزهای معماری، به ساختار سیستم آسیب می‌زند.



**2** تخلف‌های کوچک به بدهی فنی تبدیل می‌شوند  
استثنای موقت و دورزدن قواعد، به مرور انباشته می‌شوند و هزینه نگهداری را چند برابر می‌کنند.



**3** منشأ تصمیم‌ها در گفتگوها گم می‌شود  
وقتی تصمیم‌ها در پیام‌ها و گفتگوها پراکنده‌اند، منبع و منطق آن‌ها قابل بازیابی نیست.



**4** نگهداری به مهندسی معکوس رفتار تبدیل می‌شود  
به جای اتکا به طراحی و مستندات، تیم مجبور است با تحلیل کد و رفتار سیستم، منطق را حدس بزند.



بدون ثبت نیت و تصمیم‌ها، معماری فرسایش می‌یابد و پاسخ‌گویی از بین می‌رود.

چه کسی، چه زمانی، بر چه مبنایی این تصمیم را گرفت؟



# خطر تمرکز بر مسیر موفق

آنچه در دمو عالی به نظر می‌رسد، الزاماً در دنیای واقعی قابل اتکا و امن نیست.

آنچه در دمو نشان داده می‌شود  
(مسیر موفق)



- ✓ جریان ساده و ایده‌آل
- ✓ ورودی‌های معتبر و کامل
- ✓ محیط کنترل شده
- ✓ نتیجه خوشایند و سریع

VS

دنیای واقعی  
(شامل شکست، خطا و سوءاستفاده)



- ✗ ورودی‌های پیش‌بینی‌نشده و مرزی
- ✗ شرایط خطا و قطعی زیرساخت
- ✗ رفتارهای مخرب و سوءاستفاده
- ✗ وابستگی‌ها و محدودیت‌ها

1 دمو معمولاً فقط مسیر موفق را نمایش می‌دهد

سناریوهای ایده‌آل انتخاب می‌شوند و خطاها، مرزها و حالت‌های غیرمنتظره پنهان می‌مانند.



2 سیستم واقعی شامل شکست و سوءاستفاده است

کاربران اشتباه می‌کنند، سرویس‌ها از دسترس خارج می‌شوند و مهاجمین مسیرهای نادرست را هدف می‌گیرند.



3 امنیت در جزئیات و محدودیت‌ها پنهان است

مجوزها، اعتبارسنجی‌ها، نرخ محدودسازی، و ایزوله‌سازی منابع اغلب در دمو دیده نمی‌شوند.



4 کارکرد ظاهری، معادل صحت مهندسی نیست

اینکه «کار می‌کند» لزوماً به معنی «درست، پایدار، امن و قابل نگهداری» بودن سیستم نیست.



ارزیابی مهندسی باید فراتر از دمو باشد: همه مسیرها، حالت‌های خطا، سوءاستفاده‌های محتمل و محدودیت‌ها باید پوشش داده شوند.



# توسعه مبتنی بر مشخصات چیست؟

SDD رویکردی است که در آن «مشخصات» منبع رسمی رفتار سیستم است و توسعه از نیت تأییدشده و شفاف آغاز می‌شود.



## 1 SDD یک لایه حاکمیتی برای هوشواره است

مثل قانون راهنمای توسعه عمل می‌کند؛ مدل، مرزها، قواعد و کیفیت را تعیین می‌کند.



## 2 مشخصات، منبع رسمی رفتار سیستم می‌شود

هر آنچه برای ساخت، تست و پذیرش لازم است به صورت دقیق، ساختارمند و قابل ردیابی نوشته می‌شود.



## 3 ابهام پیش از تولید گسترده کد حل می‌شود

سؤالات، فرضیات و تناقض‌ها در مرحله مشخصات کشف و حل می‌شوند، نه در کد و نه در عملیات.

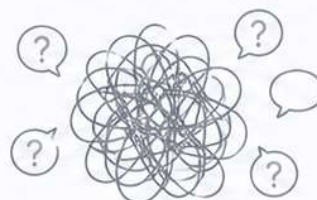


## 4 پیاده‌سازی از نیت تأییدشده مشتق می‌شود

کد، تست و پیکربندی مستقیماً از مشخصات استخراج می‌شوند؛ نه از حدس، تجربه فردی یا گفت‌وگوهای گذرا.

### SDD بدون

توسعه مبتنی بر حدس و گفتگو



کد لولیه



ابهام، تغییرات زیاد و هزینه نگهداری بالا

### SDD با

توسعه مبتنی بر مشخصات و نیت تأییدشده



#### مشخصات رسمی

- ✓ مدل دامنه و اهداف
- ✓ قواعد و محدودیت‌ها
- ✓ سناریوها و رفتار سیستم
- ✓ نیازمندی‌های غیرکارکردی
- ✓ معیارهای پذیرش و کیفیت



پیاده‌سازی، تست و پیکربندی مشتق از مشخصات



رفتار قابل پیش‌بینی، ردیابی‌پذیر و نگهداری‌پذیر



در SDD، ابتدا «چه» «چرا» و «چگونه باید کار کند» مشخص می‌شود؛ سپس «چگونه ساخته می‌شود» به صورت پایدار و تکرارپذیر دنبال می‌گردد.



# جداسازی نقش انسان و هوشواره



## نقش انسان

### معمار نیت و محدودیت‌ها

- تعیین نیت و اهداف  
چرایی و مقصد کار را مشخص می‌کند.
- تعیین محدودیت‌ها و مرزها  
قواعد، ارزش‌ها و خط قرمزها را تعیین می‌کند.
- تصمیم‌گیری درباره چرا و چه چیزی  
انسان درباره چرایی و مرزها تصمیم می‌گیرد.
- تأیید و بازبینی خروجی  
ارزیابی کیفیت و همراهی با نیت.
- مسئولیت و پاسخگویی نهایی  
انسان پاسخگو و صاحب تصمیم است.



### همکاری مؤثر



## نقش هوشواره

### موتور سریع پیاده‌سازی

- پیاده‌سازی سریع  
نیت تأییدشده را با سرعت اجرا می‌کند.
- خودکارسازی و انجام کارها  
وظایف را به صورت خودکار انجام می‌دهد.
- تولید و پردازش خروجی  
کد، متن، تحلیل و سایر خروجی‌ها را تولید می‌کند.
- تکرار و بهبود سریع  
با بازخورد، نسخه‌های بهتر ارائه می‌دهد.
- بدون حدس نیت گمشده  
عامل نباید نیت گمشده را حدس بزند.



### انسان، معمار نیت و محدودیت‌ها است

انسان درباره چرایی و مرزها تصمیم می‌گیرد.



### هوشواره، موتور سریع پیاده‌سازی است

عامل نباید نیت گمشده را حدس بزند.

# وارونگی قدرت میان مشخصات و کد

SDD، مشخصات منبع رسمی رفتار سیستم است و کد باید از آن پیروی کند، نه برعکس.

## در گذشته (رویکرد سنتی)



### کد در جایگاه حقیقت نهایی

- ❌ کد موجود، منبع رفتار سیستم است
- ❌ مستندات اغلب عقب افتاده و ناقص اند
- ❌ تغییر رفتار ممکن است بی صدا رخ دهد
- ❌ ردیابی چرایی تصمیمها دشوار است



**نتیجه:** کد مرجع اصلی است و رفتار واقعی ممکن است از نیت اولیه فاصله بگیرد.

## SDD در زرویکرد (Specification-First)



### مشخصات در جایگاه مرجع رسمی

- ✅ مشخصات منبع رسمی و بالاترین مرجع است
- ✅ کد باید مطابق مشخصات باشد
- ✅ هرگونه ناسازگاری باید آگاهانه حل شود
- ✅ تغییرات رفتار باید قابل ردیابی و بازبینی باشد



**نتیجه:** مشخصات مرجع است؛ کد باید آن را پیاده سازی و رعایت کند.

## اصول کلیدی وارونگی قدرت



### ۱. مرجعیت مشخصات

مشخصات، معیار نهایی برای ارزیابی رفتار سیستم است.



### ۲. حل آگاهانه ناسازگاری

هر تضاد میان کد و مشخصات باید آگاهانه، مستند و قابل ردیابی باشد.



### ۳. ردیابی تصمیمها

هر تغییر باید چرایی، منبع تصمیم و اثرات آن مشخص باشد.



### ۴. عدم پذیرش تغییر خاموش

هیچ تغییری در رفتار سیستم بدون به روزسانی مشخصات پذیرفته نیست.



در SDD، کد خدمتگزار مشخصات است، نه صاحب آن. قدرت تصمیم گیری نزد انسان باقی می ماند و هوش آوره آن را اجرا می کند.



# کد به عنوان مصنوع مشتق شده

در SDD، کد محصول نهایی نیست؛ نتیجه طبیعی و تکرارپذیر نیت دقیق و ساختاریافته است.

1



کد ترجمه‌ای از نیت ساختاریافته است  
مشخصات رسمی، مدل داده، قوانین و  
جریان‌ها، به کد تبدیل می‌شوند.

2



ارزش اصلی در تعریف دقیق مسئله  
سرمایه واقعی، فهم درست، مرزبندی و  
توافق روی «چه چیزی» و «چرا» است.

3



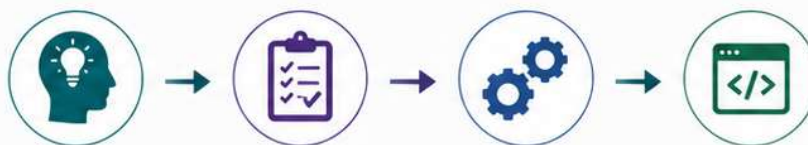
کد همچنان ضروری، اما بازتولیدپذیرتر می‌شود  
با مشخصات دقیق، کد قابل بازتولید، جایگزین  
و بازسازی توسط عامل‌هاست.

4



دارایی اصلی، نیت کنترل‌شده سازمان است  
دارایی پایدار سازمان، مشخصات، تصمیم‌ها و  
محدودیت‌های نسخه‌دار و قابل ردیابی است.

## مسیر تبدیل نیت به کد



نیت و هدف

نیازها، اهداف،  
محدودیت‌ها،  
فرض‌ها

مشخصات ساختاریافته

مدل داده، قوانین کسب‌وکار،  
جریان‌ها، قراردادها،  
معیارهای پذیرش

تولید و پیاده‌سازی

تولید کد، پیکربندی،  
پیکارچه‌سازی، تست

کد (مصنوع مشتق‌شده)

ترجمه عملیاتی نیت،  
قابل اجرا، تست و  
استقرار



با تغییر مشخصات، کد به صورت کنترل‌شده و قابل ردیابی بازتولید می‌شود.

## نتایج کلیدی



قابلیت اعتماد بیشتر

رفنار سیستم از نیت شفاف و کنترل‌شده  
مشتق می‌شود.



تغییر ایمن و پایدار

تغییر در نیت، منجر به تغییر کنترل‌شده  
در خروجی (کد) می‌شود.



قابلیت ردیابی و حسابرسی

هر خط کد، به تصمیم و مشخصه‌ای  
مشخص قابل بازگشت است.



هم‌افزایی انسان و هوش‌واره

انسان نیت را هدایت می‌کند؛ هوش‌واره  
کد را با سرعت و دقت تولید می‌کند.



در SDD، مزیت رقابتی شما «کد بیشتر» نیست؛ بلکه «نیت دقیق‌تر و کنترل‌شده‌تر» است.



# مشخصات زنده، نه مستندات ایستا

وقتی سیستم تغییر می‌کند، مشخصات هم باید همراه آن تکامل یابد.



مشخصات باید زنده، دقیق و همگام با سیستم باشد؛ اسناد بیات، ریسک تولید خطا را به‌طور چشمگیری افزایش می‌دهد.

# مشخصات قابل اجرا چه دارد؟



مشخصات قابل اجرا، فقط شرح مسئله نیست؛ نقشه رفتار سیستم است.

# ابهام باید قابل مشاهده باشد

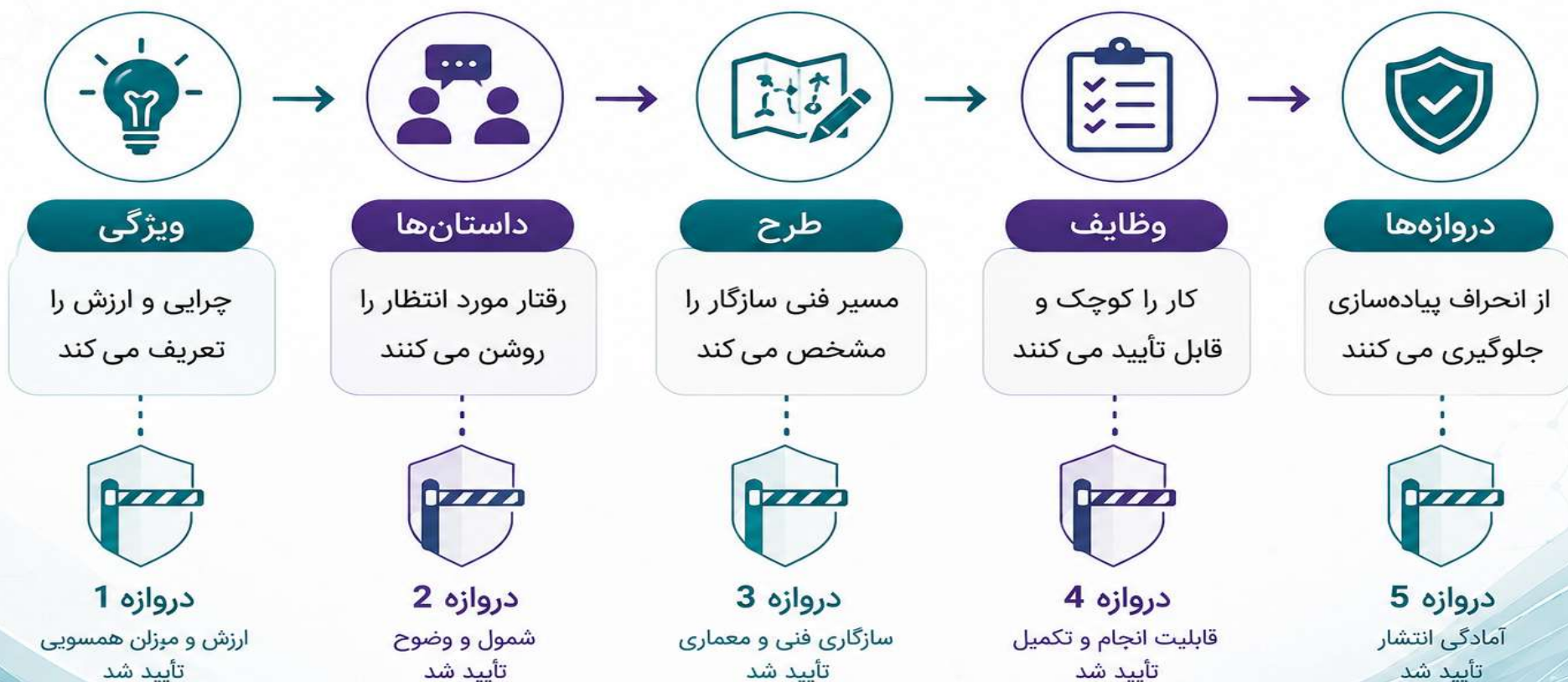


ابهام پنهان، ریسک پنهان است. | ابهام قابل مشاهده، تصمیم‌گیری ایمن ایجاد می‌کند.



اطلاعات ناقص باید سؤال ایجاد کند یا جریان کار را متوقف سازد، نه حدس و گمان خاموش.

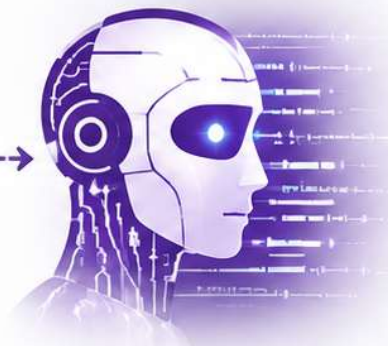
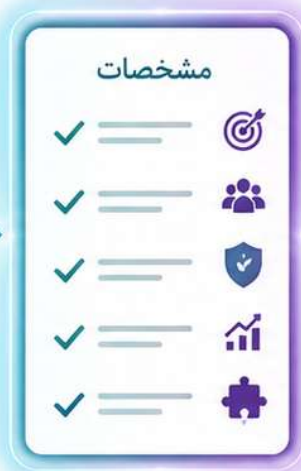
# جریان عملی و دروازه‌های SDD



# آینده مهندسی، معماری نیت است



انسان  
نیت را معماری  
می‌کند



هوشواره  
کد می‌سازد



مشخصات، زبان مشترک انسان و ماشین است



سرعت بدون وضوح،  
آشفته‌گی را تقویت می‌کند



ارتباط دقیق به  
یک توانایی فنی تبدیل می‌شود



مشخصات،  
زبان مشترک انسان و ماشین است



هوشواره کد می‌سازد؛  
انسان نیت را معماری می‌کند



هرچه عامل سریع‌تر شود،  
مشخصات مهم‌تر می‌شود

# جمع بندی

SDD، مهندسی نرم افزار در عصر عامل‌ها



## مشخصات برتر از کد

مشخصات، منیع حقیقت است و باید همیشه جلوتر باشد.



## عامل‌ها سریع‌اند

هرچه عامل سریع‌تر شود، ومشخصات مهم‌تر می‌شود.



## ارتباط دقیق، کلید موفقیت

مشخصات، زبان مشترک انسان و ماشین است.



## جریان کنترل شده

جریان عملی و دروازه‌های SDD از انحراف جلوگیری می‌کنند.



## نیت، دارایی اصلی سازمان

کد قابل بازتولید است؛ نیت، مزیت پایدار است.

### جریان عملی و دروازه‌های SDD



#### ویژگی

چرایی و ارزش را تعریف می‌کند.



#### داستان‌ها

رقنار مورد انتظار را روشن می‌کنند.



#### طرح

مسیر فنی سازگار را مشخص می‌کند.



#### وظایف

کار را کوچک و قابل تأیید می‌کنند.



#### دروازه‌ها

از انحراف پیاده‌سازی جلوگیری می‌کنند.



### ★ نکته‌ی کلیدی

در SDD، ما ابتدا نیت را معماری می‌کنیم و سپس هوش‌واره آن را با سرعت به کد قابل اجرا تبدیل می‌کنند.

### آنچه SDD به ما می‌دهد



وضوح و همترازی در تیم و با عامل‌ها



کاهش تغییرات ناخواسته و بازگشت‌های پرهزینه



قابلیت، ردیابی، نسخه‌پذیری و تصمیم‌های آگاهانه



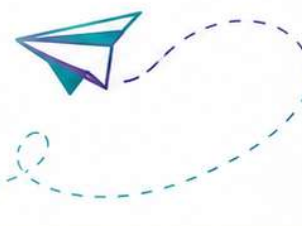
تمرکز بر ارزش واقعی، نه صرفاً نوشتن کد

”آینده متعلق به تیم‌هایی است که نیت خود را شفاف، ساختاریافته و قابل اجرا می‌کنند.“

# بیایید عمل کنیم!

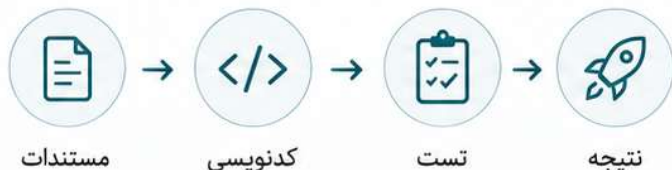


اکنون یک مثال ساده را به دو روش متفاوت پیاده‌سازی می‌کنیم  
و تفاوت نتیجه را با هم بررسی می‌کنیم.



## روش سنتی

کد → کد



VS

## روش مبتنی بر SDD

نیت → مشخصات → کد



## مشاهده یک نمونه کوتاه پیاده‌سازی

به روش سنتی و روش مبتنی بر SDD



# پرسش و پاسخ

سؤالات، دیدگاه‌ها و تجربه‌های شما برای ما ارزشمند است.



## سؤال دارید؟

هر سؤالی درباره‌ی مفاهیم، روش‌ها یا ابزارها دارید بپرسید.



## دیدگاهی دارید؟

تجربه‌ها و دیدگاه‌های شما می‌تواند بحث را غنی‌تر کند.



## نمونه‌ای به اشتراک دارید؟

مثال‌ها و نمونه‌های واقعی شما، یادگیری را ماندگارتر می‌کند.



بیایید با هم یاد بگیریم و بهتر بسازیم.

